

CSA3023/CSE3023/CSM3023 WEB-BASED APPLICATION DEVELOPMENT

Lab Modules



WAN NURAL JAWAHIR HJ WAN YUSOF, RABIEI MAMAT,
MOHD ARIZAL SHAMSIL MAT RIFIN
& MUHAMMAD SYAFFIQ HAZARI

Contents

List of Figures

iv

1	DEVELOPMENT ENVIRONMENT SETUP FOR JAVA WEB APPLICATIONS	1
1.1	Objectives	1
1.2	Introduction	1
1.3	Software Requirements	2
1.4	Task 1: Install Java Development Kit (JDK)	2
1.4.1	Step 1: Download JDK	2
1.4.2	Step 2: Install JDK	2
1.4.3	Step 3: Verify JDK Installation	2
1.5	Task 2: Install XAMPP	3
1.5.1	Step 1: Download XAMPP	3
1.5.2	Step 2: Install XAMPP without Tomcat	4
1.5.3	Step 3: Verify XAMPP Installation	9
1.6	Task 3: Change the Default Root Password of MySQL	11
1.6.1	Step 1: Start Apache and MySQL Services	11
1.6.2	Step 2: Run Command	12
1.7	Task 4: Tomcat 9 Installation	18
1.7.1	Step 1: Download Apache Tomcat 9	18
1.7.2	Step 2: Extract Apache Tomcat 9	19
1.7.3	Step 3: Start Tomcat Server	19
1.7.4	Step 4: Verify Installation	20
1.8	Task 5: NetBeans IDE Installation	21
1.8.1	Step 1: Download Apache NetBeans 29	21
1.8.2	Step 2: Extract and Install NetBeans 29	21
1.8.3	Step 3: Configure NetBeans 29	23
1.8.4	Step 5: Configure Tomcat in NetBeans	23
1.9	Task 6: Writing a Simple Java Servlet	26
1.9.1	Step 1: Create New Project	26
1.9.2	Step 2: Create New Servlet	29
1.9.3	Step 3: Run Servlet	32
1.10	Task 7: Writing a Simple JSP Program	34
1.10.1	Step 1: Create a New Project	34
1.10.2	Step 2: Create a New JSP File	36
1.10.3	Step 3: Run JSP File	39
1.11	Task 8: Use Java Reference Datatype/Class Wrapper in JSP	41
1.11.1	Step 1: Create a New JSP File	41

1.12	Exercise	43
2	SERVLET: DATA SHARING AND DATABASE MANAGEMENT	45
2.1	Objectives	45
2.2	Introduction	46
2.3	Software Requirements	46
2.4	Task 1: Data Sharing in Servlet	46
2.4.1	Step 1: Project Setup	46
2.4.2	Step 2: Create HTML Login Page	48
2.4.3	Step 3: Create a Login Servlet	48
2.4.4	Step 4: Create an Account Servlet	51
2.4.5	Step 5: Run the Project	52
2.5	Task 2: Creating A Table in MySQL Database	53
2.5.1	Step 1: Start Apache and MySQL	53
2.5.2	Step 2: Create a New Database	53
2.5.3	Step 3: Create a New MySQL Table	53
2.6	Task 3: Setting the Environment of Web Application for Database Connection	54
2.6.1	Step 1: Create a New Web Application Project	54
2.6.2	Step 2: Prepare the MySQL Connector	55
2.7	Task 4: Using Servlets for Database CRUD Operations	57
2.7.1	Step 1: Create HTML Form for Data Entry	57
2.7.2	Step 2: Create the Insert Servlet (Create)	58
2.7.3	Step 3: Create the View Servlet (Read)	59
2.7.4	Step 4: Create the Delete Servlet (Delete)	62
2.7.5	Step 5: Create the Update Servlet (Update)	63
2.8	Task 5: Using Servlets, DAO and Java Beans for Database CRUD Operations	65
2.8.1	Step 1: Copy and Rename the Project	66
2.8.2	Step 2: Create the User JavaBean (Model)	66
2.8.3	Step 3: Create the Data Access Object (UserDAO)	68
2.8.4	Step 4: Modify the Existing Servlets (Controller)	72
2.9	Exercise	77
3	JSP: SCRIPTLET, PAGE DIRECTIVE & INCLUDE DIRECTIVE	79
3.1	Objectives	79
3.2	Introduction	79
3.3	Software Requirements	80
3.4	Task 1: Passing Data from Main JSP's Page to Other JSP's Page	80
3.4.1	Step 1: Create A New JSP Page	80
3.4.2	Step 2: Write an HTML Markup	81
3.5	Task 2: Using Mathematics Operations in JSP	84
3.5.1	Step 1: Create A New JSP Page	85
3.5.2	Step 2: Write an HTML Markup	85
3.5.3	Step 3: Run the <code>Calculator.jsp</code>	86
3.6	Task 3: Populate an Array Values into HTML's Table	89
3.6.1	Step 1: Create A New JSP Page	89
3.6.2	Step 2: Write An HTML Markup	89
3.6.3	Step 3: Run the <code>populateArray.jsp</code> File	90

3.7	Task 4: Perform Calculation of Car Loan	90
3.7.1	Step 1: Create A New HTML File	90
3.7.2	Step 2: Create An HTML Markup	91
3.7.3	Step 3: Create <code>processCalculateCarLoan.jsp</code>	92
3.8	Task 5: Using JSP Page Directive to Call Java API	93
3.9	Task 6: Use JSP Include directive for JSP Page	98

List of Figures

1.1	XAMPP Download Platform	3
1.2	XAMPP Setup Wizard Startup Screen	5
1.3	XAMPP Select Components	5
1.4	XAMPP Installation Directory	6
1.5	XAMPP Language	7
1.6	XAMPP Installation Ready	7
1.7	XAMPP Installation Progress	8
1.8	XAMPP Installation Finish	8
1.9	XAMPP Control Panel	9
1.10	XAMPP Control Panel Running	10
1.11	XAMPP Dashboard	10
1.12	phpMyAdmin Interface	11
1.13	XAMPP Control Panel-Click Shell	11
1.14	Verify Password	12
1.15	Error Page	13
1.16	Apache Tomcat9 Download Page	18
1.17	Start Tomcat Server	19
1.18	Apache Tomcat Homepage	20
1.19	Apache NetBeans 29 Download Page	21
1.20	Extract netbeans-29-bin.zip	22
1.21	Netbean Executable File	22
1.22	Netbean IDE GUI	23
1.23	Configure Server	24
1.24	Add Server	24
1.25	Add Server	25
1.26	Add Server	25
1.27	Add Server	26
1.28	Create New Project	27
1.29	Create New Web Application	27
1.30	Enter Project Details	28
1.31	Create New Servlet	29
1.32	Name the Servlet	30
1.33	Configure Servlet Deployment	30
1.34	HelloServlet.java File	31
1.35	<code>processRequest()</code> Method	31
1.36	Modified <code>processRequest()</code> Method	31
1.37	Run the Servlet	32

1.38	Dialog Box Call to /HelloServlet	32
1.39	Output	33
1.40	Modified <code>doGet()</code> Method	33
1.41	Dialog Box with Passing Parameter	34
1.42	Output	34
1.43	Create New Web Application	35
1.44	Enter Project Details	36
1.45	Configure the Server	36
1.46	Create a New JSP File	37
1.47	Enter Name and Location	37
1.48	Modified JSP File	38
1.49	Compile JSP File	38
1.50	Compile JSP File	38
1.51	Run JSP File	39
1.52	Output	40
1.53	Start/Stop Apache Tomcat	40
1.54	Create <code>useJavaObject.jsp</code>	41
1.55	<code>useJavaObject.jsp</code>	42
1.56	Output	43
2.1	Create New Servlet	49
2.2	Create Login Servlet	49
2.3	Configure servlet deployment	50
2.4	Creating the “CRUDusingServlet” Project	55
2.5	Downloading MySQL Connector/J	55
2.6	Extracting the MySQL Connector/J Archive	56
2.7	Adding External Library to Project	56
2.8	Importing MySQL Connector/J Library	57
3.1	Create new JSP	81
3.2	New JSP	81
3.3	<code>memberRegister.jsp</code> Interface	82
3.4	Filling input <code>memberRegister.jsp</code>	84
3.5	Output in <code>memberProcessing.jsp</code>	84
3.6	Create new JSP	85
3.7	New JSP	85
3.8	<code>calculator.jsp</code> Interface	86
3.9	Key-in Filename	89
3.10	Output from Array	90
3.11	Creating New HTML File	91
3.12	Interface Output for Car Loan Calculation HTML	91
3.13	Input for Car Loan Calculation	92
3.14	Output for Car Loan Calculation	93
3.15	New JSP	93
3.16	Key-in Filename	94
3.17	Save All	96
3.18	Run	96
3.19	Success Message	96

3.20	ArrayList Output	97
3.21	Tomcat Before Run	97
3.22	Tomcat After Run	97
3.23	Tomcat Success Message	98
3.24	mainPage.jsp	98
3.25	headerPage.jsp Output	98
3.26	footerPage.jsp Output	99
3.27	mainPage.jsp Output	99

Lab Module 3

JSP: SCRIPTLET, PAGE DIRECTIVE & INCLUDE DIRECTIVE

3.1 Objectives

The primary goal of this lab is to familiarize students with dynamic web page development using JavaServer Pages (JSP). By the end of this session, students should be able to:

1. **Implement Data Transfer:** Demonstrate the ability to pass user input from a main JSP page to a processing page using the `request.getParameter()` method.
2. **Apply Mathematical Operations:** Use Java Scriptlets to perform server-side calculations (e.g., basic calculator functions and car loan interest) based on user-provided data.
3. **Manage Data Structures:** Populate HTML tables dynamically by reading and iterating through Java arrays and `ArrayList` objects.
4. **Utilize JSP Directives:** Apply the page directive to import Java APIs and the include directive to create modular web pages with reusable headers and footers.

3.2 Introduction

JavaServer Pages (JSP) is a server-side technology used to create dynamic, platform-independent web applications. Unlike static HTML, JSP allows developers to insert Java code directly into web pages using Scriptlets (`<% ... %>`), enabling the page to process logic before being sent to the client's browser.

This lab focuses on the core components of JSP architecture, specifically how the web server (such as Apache Tomcat) handles client requests, processes data through Java logic, and generates a dynamic response. Students will explore the use of Directives to control page settings and Actions to include external resources, which are essential for building scalable and maintainable web interfaces.

3.3 Software Requirements

To successfully complete the tasks outlined in this manual, the following software environment is required:

- **Java Development Kit (JDK):** Version 8 or higher (required to compile Java Scriptlets).
- **Integrated Development Environment (IDE):** NetBeans
- **Web Server:** Apache Tomcat 8.0 or GlassFish Server 4.1.
- **Web Browser:** Any modern browser (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge) for testing and viewing output.

3.4 Task 1: Passing Data from Main JSP's Page to Other JSP's Page

Objective: To demonstrate the use of `request.getParameter` ("fieldName") for passing input from one JSP's page to another JSP's page.

3.4.1 Step 1: Create A New JSP Page

1. Create new Project namely Lab2.
2. To create a JSP's page, right click Lab2 → New → JSP.
3. Key-in File Name: `memberRegister`.
4. Click **Finish** button.
5. The source code for `memberRegister.jsp` will appear.



Figure 3.1: Create new JSP

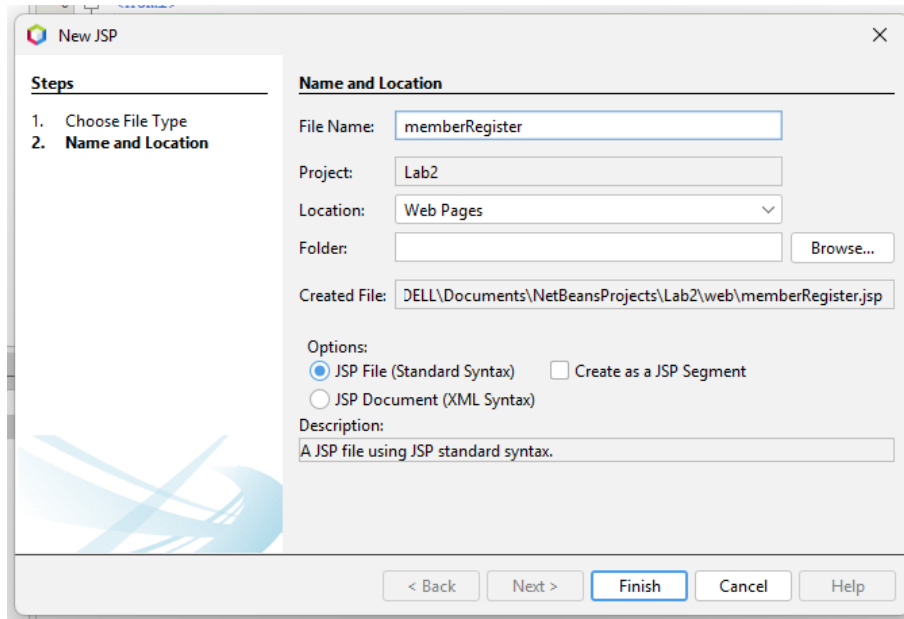


Figure 3.2: New JSP

3.4.2 Step 2: Write an HTML Markup

1. Write an HTML markup to produce the HTML form.

```
<body>
  <h1>Passing data from main JSP page to other JSP page</h1>

  <form id="memberFrm" action="memberProcessing.jsp" method="post"
onsubmit="return checkICNo()">
    <fieldset>
      <legend>Member Registration</legend>

      <label for="icno">IC. No</label>
      <input type="text" id="icno" name="my_icno" size="15"
placeholder="E.g. 930123034567"><br/>

      <label for="icno">Name</label>
      <input type="text" id="name" name="my_name" size="45"
```

```

placeholder="Key-in your name"><br/>

<p>
    <input type="submit" id="btnSubmit" value="Submit">
    <input type="reset" id="btnCancel" value="Cancel">
</p>

</fieldset>
</form>

&copy;2026-Syafiq
</body>

```

2. Save and compile memberRegister.jsp file.
3. Run the memberRegister.jsp file and you should get the interface as below:



Figure 3.3: memberRegister.jsp Interface

4. Repeat step 1 and step 2.
5. Key-in File Name: memberProcessing.
6. Click Finish button.
7. Source code for memberProcessing.jsp will appear.
8. Write a HTML code.

```

<!DOCTYPE html>
<html>
<head>
    <meta http-equiv="Content-Type" content="text/html"; charset="UTF-8">
    <title>Lab 3 - Task 1</title>

```

```

</head>
<body>
    <h2>Passing data from main JSP page to another JSP page</h2>
</body>
</html>

```

9. Add additional HTML's tag and Java Scriptlet to retrieve the value from main's form.

```

<fieldset>
<%
    //define variables..
    String myIC = null;
    String myName = null;

    <!-- use request.getParameter() method to retrieve data from the form
    in memberRegister.jsp -->
    myIC = request.getParameter("my_icno");
    myName = request.getParameter("my_name");
%>

<!-- display the output -->
<p>Thank you for registering in this event..!</p>
<p>This is your details:</p>
<p>IC No : <%= myIC %></p>
<p>Name : <%= myName %></p>

</fieldset>

```

10. Compile memberProcessing.jsp file.
11. Run the memberRegister.jsp file and fill-up the input.
12. Click Submit button to send the request.

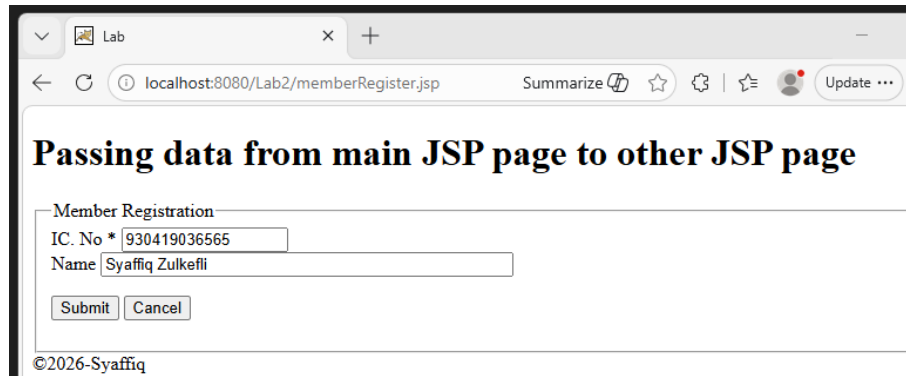


Figure 3.4: Filling input `memberRegister.jsp`

13. These inputs will be sent to `memberProcessing.jsp` page and produce the following page.



Figure 3.5: Output in `memberProcessing.jsp`

Reflection

1. How do you want to submit specific information from one form to next form?
2. What happened if the field name you specify in `request.getParameter("field_name")` in second page is different from the field name you defined in first page?

3.5 Task 2: Using Mathematics Operations in JSP

Objective: To demonstrate the use of `request.getParameter("Mathematics operations")` in JSP's page.

3.5.1 Step 1: Create A New JSP Page

1. Go to Project Lab2.
2. To create a JSP's page, right click Lab2 → New → JSP.

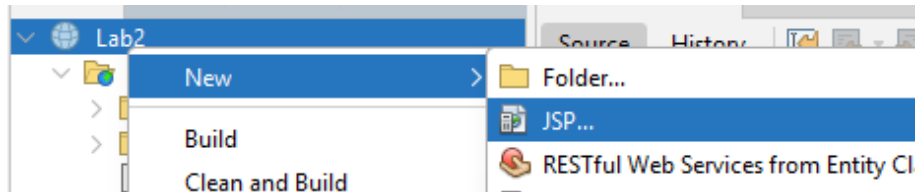


Figure 3.6: Create new JSP

3. Key-in File Name: Calculator.

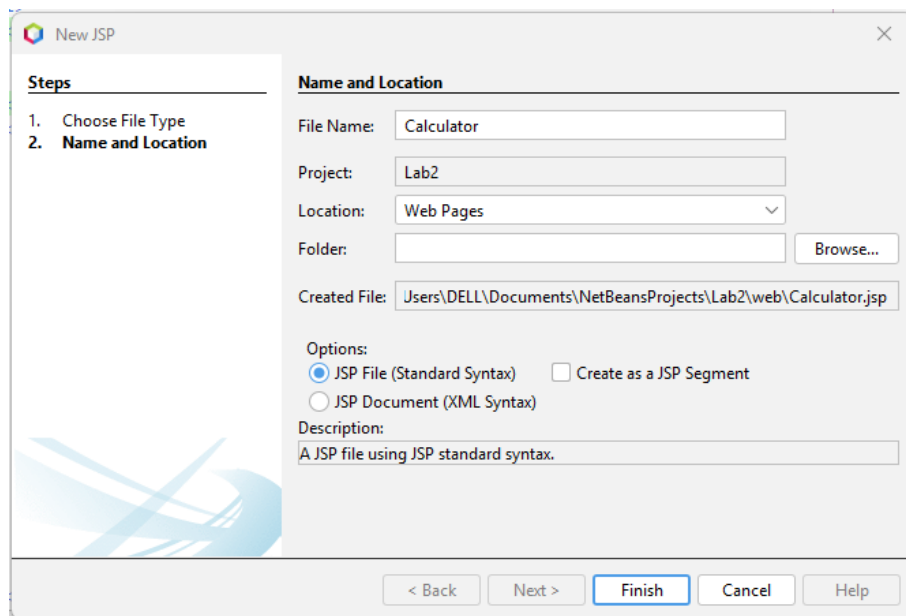


Figure 3.7: New JSP

4. Click Finish button.
5. The source code for Calculator.jsp will appear.

3.5.2 Step 2: Write an HTML Markup

1. Write an HTML's markup to produce an HTML form.

```
<body bgcolor="#87CEEB" text="grey">
  <center>
    <h2>Basic calculator program in JSP</h2>
```

```

<form method="get" name="f1">
    <input type="text" size="20" name="operand1" value="" />

    <select name="op" size="1">
        <option value="0">+</option>
        <option value="1">-</option>
        <option value="2">*</option>
        <option value="3">/</option>
        <option value="4">%</option>
    </select>

    <input type="text" size="20" name="operand2" value="" />
    <br/><br/>

    <input type="submit" value="Calculate"/>
</form>
</center>
</body>

```

2. Save and compile Calculator.jsp file.

3.5.3 Step 3: Run the Calculator.jsp

1. Run the Calculator.jsp file and you should get the interface as below:

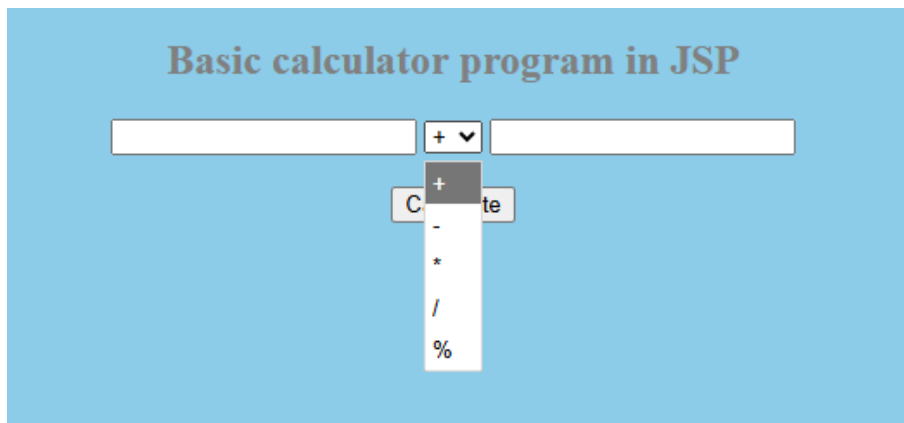


Figure 3.8: calculator.jsp Interface

2. Add additional HTML's tag and Java scriptlet to retrieve the value from users.

```
String num1 = "0", num2 = "0";
int result = 0;
String op = "+";

char opchar = op.charAt(0);
if (request.getParameter("op") != null) {
    op = request.getParameter("op");
    opchar = op.charAt(0);

    num1 = request.getParameter("operand1");
    num2 = request.getParameter("operand2");

    switch(opchar){
        case '0': result = Integer.parseInt(num1) +
            Integer.parseInt(num2);
            break;
        case '1': result = Integer.parseInt(num1) -
            Integer.parseInt(num2);
            break;
        case '2': result = Integer.parseInt(num1) *
            Integer.parseInt(num2);
            break;
        case '3': result = Integer.parseInt(num1) /
            Integer.parseInt(num2);
            break;
        case '4': result = Integer.parseInt(num1) %
            Integer.parseInt(num2);
            break;
    }
}
```


3. Further, add additional Java Scriplet to HTML's tag as below:

```
<body bgcolor="#87CEEB" text="grey">
  <center>
    <h2>Basic calculator program in JSP</h2>
    <form method="get" name="f1">
      <input type="text" size="20" name="operand1"
        value="<%= num1 %>" />

      <select name="op" size="1">
        <option value="0">+</option>
        <option value="1">-</option>
        <option value="2">*</option>
        <option value="3">/</option>
        <option value="4">%</option>
      </select>

      <input type="text" size="20" name="operand2"
        value="<%= num2 %>" />
      <br><br>
      <input type="submit" value="Calculate"/>
    </form>
  </center>

  Result = <%= result + " " %>
</body>
```

4. Compile Calculator.jsp file.

5. Run the Calculator.jsp file and test the calculator.

Reflection

1. How do you want to submit specific information from one form to next form?
2. What happened if the field name you specify in request.getParameter("field_name") in second page is different from the field name you defined in first page?

3.6 Task 3: Populate an Array Values into HTML's Table

Objective: Read Java array and populate it into HTML's table.

3.6.1 Step 1: Create A New JSP Page

1. Go to Project Lab2.
2. To create a JSP's page, right click Lab2 → New → JSP.
3. Key-in File Name: `populateArray`.

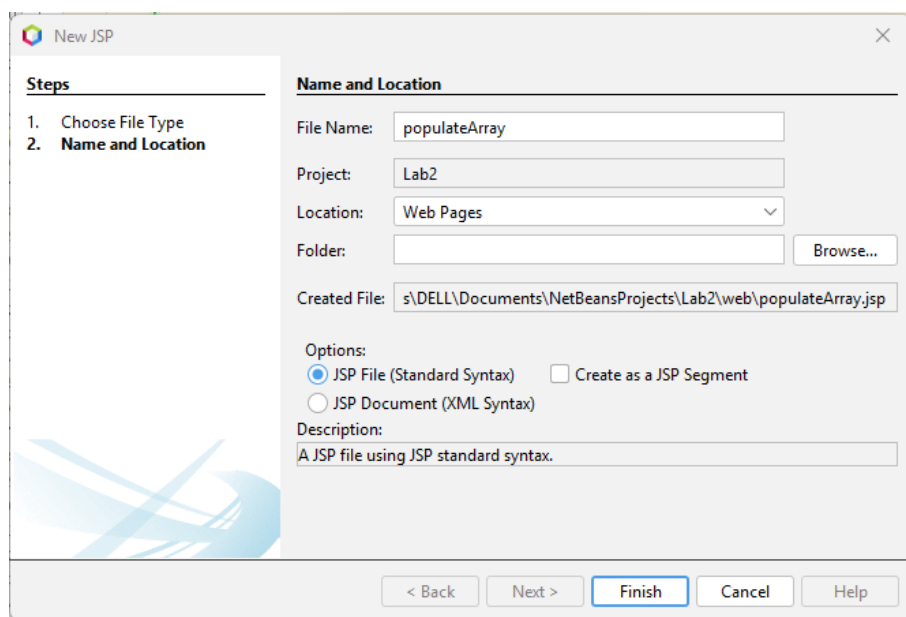


Figure 3.9: Key-in Filename

4. Click **Finish** button.

3.6.2 Step 2: Write An HTML Markup

1. Prepare standard HTML's a markup for page `populateArray.jsp`.
2. Write a Java Scriplet and store the following information into an array;

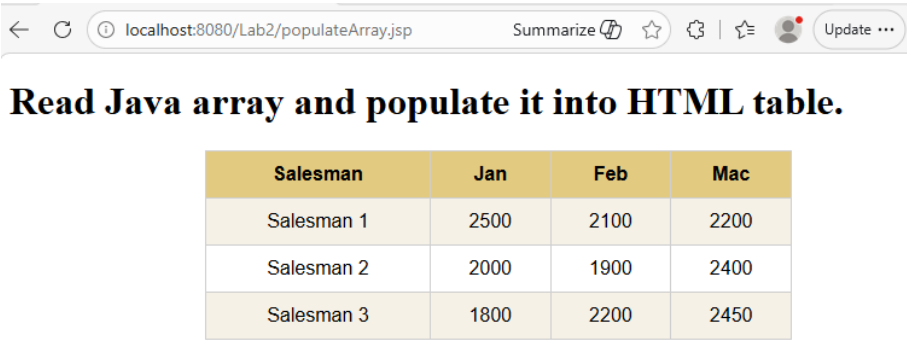
Table 3.1: Salesman's Sale

Salesman	Jan	Feb	Mar
Salesman 1	2,500	2,100	2,200
Salesman 2	2,000	1,900	2,400
Salesman 3	1,800	2,200	2,450

3. Read the array and populate its value into HTML's table.
4. Save and compile populateArray.jsp file.

3.6.3 Step 3: Run the populateArray.jsp File

1. Run the populateArray.jsp file and you should get the interface as below:



Read Java array and populate it into HTML table.

Salesman	Jan	Feb	Mac
Salesman 1	2500	2100	2200
Salesman 2	2000	1900	2400
Salesman 3	1800	2200	2450

©2026-Syaffiq

Figure 3.10: Output from Array

Reflection

1. Write a sample syntax to declare 2D Java array.
2. Define a sequence of steps on how you accomplish Task 7.
3. What is the difference between HTML's page and JSP's page.

3.7 Task 4: Perform Calculation of Car Loan

Objective: Passing input to next page for further processing.

3.7.1 Step 1: Create A New HTML File

1. Go to Project Lab2.
2. To create an HTML page, right click Lab2 → New → HTML.
3. Key-in File Name: calculateCarLoan.

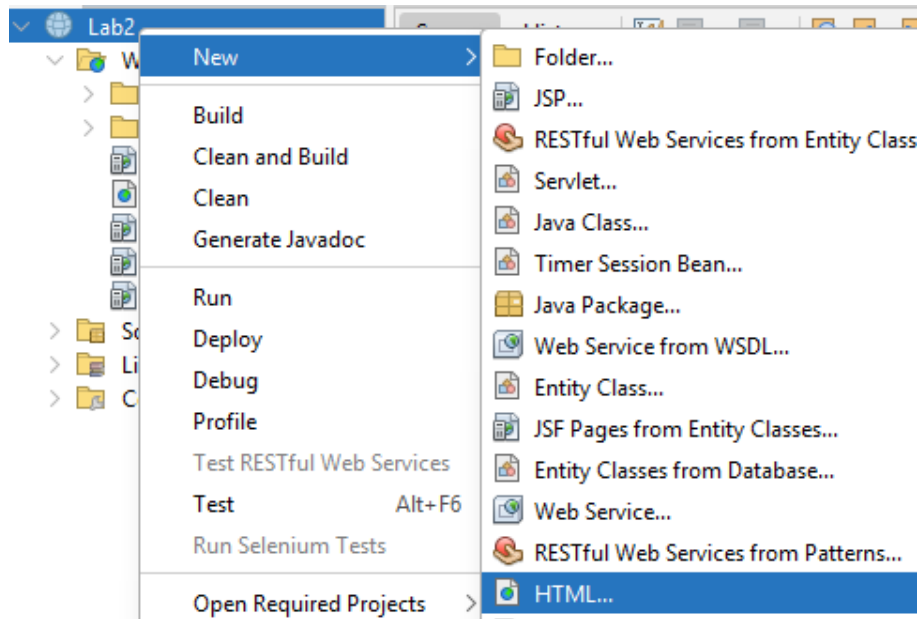


Figure 3.11: Creating New HTML File

3.7.2 Step 2: Create An HTML Markup

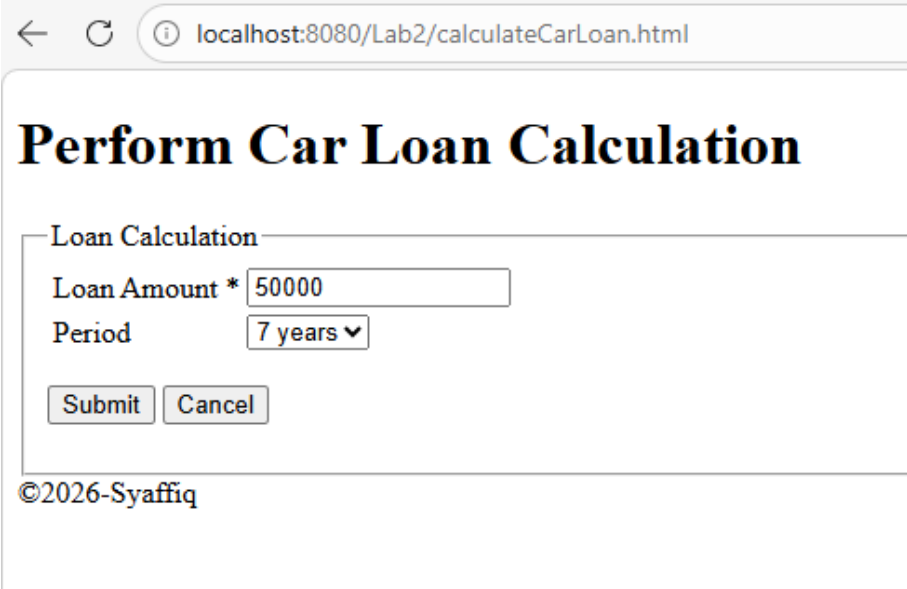
1. Create a standard HTML markup for form.
2. In your form, create two (2) fields;
 - Loan Amount
 - Loan Period
3. Save `calculateCarLoan.html` and run the file.
4. You will get the following output:

 A screenshot of a web browser displaying a form titled 'Perform Car Loan Calculation'. The browser's address bar shows 'localhost:8080/Lab2/calculateCarLoan.html'. The form has a title 'Loan Calculation' and contains two input fields: 'Loan Amount *' with a text input containing 'Amount', and 'Period' with a dropdown menu showing '3 years'. Below the inputs are 'Submit' and 'Cancel' buttons. A copyright notice '©2026-Syaffiq' is visible at the bottom left of the form.

Figure 3.12: Interface Output for Car Loan Calculation HTML

3.7.3 Step 3: Create `processCalculateCarLoan.jsp`

1. Create JSP's file and rename the file as `processCalculateCarLoan`.
2. Construct the logic for calculating car loan and display the result.
3. Save and compile `processCalculateCarLoan.jsp`.
4. Run `calculateCarLoan.html` file and fill-up the input.
5. Then, submit your result.
6. You should get the following output;



The screenshot shows a web browser window with the address bar displaying `localhost:8080/Lab2/calculateCarLoan.html`. The page title is "Perform Car Loan Calculation". Below the title, there is a form titled "Loan Calculation". The form contains two input fields: "Loan Amount *" with the value "50000" and "Period" with a dropdown menu showing "7 years". Below these fields are two buttons: "Submit" and "Cancel". At the bottom of the page, there is a copyright notice: "©2026-Syaffiq".

Figure 3.13: Input for Car Loan Calculation

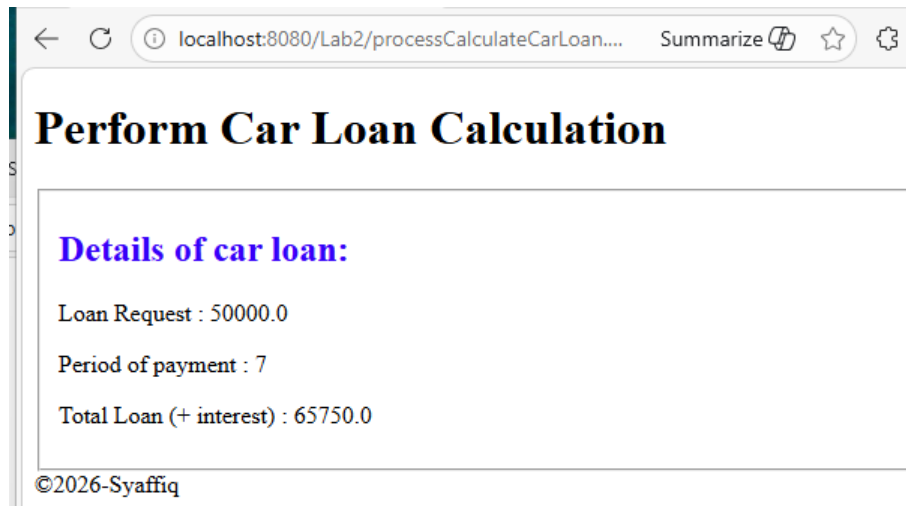


Figure 3.14: Output for Car Loan Calculation

Reflection

1. How you want to retrieve data from previous page?
2. Where the construction of logic occur for calculating Total Loan (+ interest)?

3.8 Task 5: Using JSP Page Directive to Call Java API

Objective: Use JSP page directive elements to call certain Java API.

1. Create a new JSP's file.

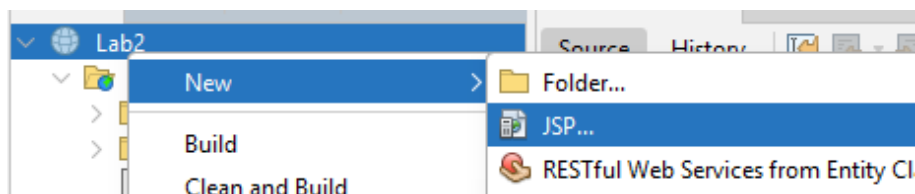


Figure 3.15: New JSP

2. Type file name as `ArrayList`.
3. Click **Finish** button.
4. Type title as *Use Java ArrayList*.
5. Type header1 as *Use JSP Page Directive*.

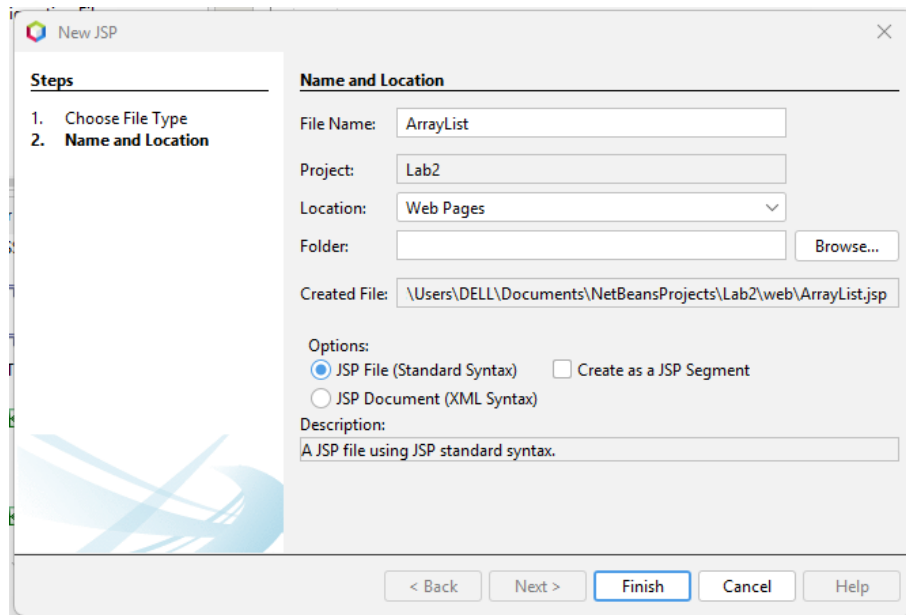


Figure 3.16: Key-in Filename

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Use Java ArrayList</title>
  </head>
  <body>
    <h1>Use JSP Page Directive</h1>
    <br>
    <footer>&copy;2026-Syaffiq</footer>
  </body>
</html>
```

6. In order to use Java ArrayList's object, we need to use JSP Page Directive and import the related API.

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.ArrayList"%>
```

7. In order to use Java syntax, create a Java Scriptlet notation.

```
<body>
  <h1>Use Java Page Directive</h1>
```

```
<%  
  
%>
```

8. Create an object `ArrayList` to store a list of student name

```
// create ArrayList object  
ArrayList<String> studentList = new ArrayList<String>();
```

9. Add the following name to `ArrayList`'s object.

- Mohamad Azam
- Peter Chong
- Rahimah Mansor
- Sri Devi
- Ng Hue Ween
- S. Nagarajan

```
// store student name  
studentList.add(0, "Mohamad Azam");  
studentList.add(1, "Peter Chong");  
studentList.add(2, "Rahimah Mansor");  
studentList.add(3, "Sri Devi");  
studentList.add(4, "Ng Hue Ween");  
studentList.add(5, "S. Nagarajan");
```

10. Display the number of records for an `ArrayList`'s object.

```
// display the number of records  
out.println("<p>The number of records in Arraylist is " +  
            studentList.size() + "</p>");
```

11. Finally, populate the list of students.

```
// populate list of students  
for (int i=0; i<studentList.size(); i++)  
    out.println("<p>Record " + (i+1) + " is " +  
                studentList.get(i) + "</p>");
```

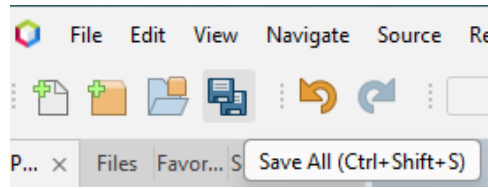



Figure 3.17: Save All

12. Click *SaveAll* icon.

13. Click *Run* icon.

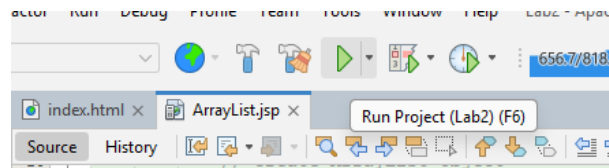


Figure 3.18: Run

14. You will get notification message the the bottom of Netbeans IDE with the green colour.



Figure 3.19: Success Message

15. Output will appear in web browser.

16. Before running any JSP's files for first time upon opening your Netbeans IDE, you need to start your web server (i.e; Apache Tomcat).

17. To perform this, go to **Services** → **Servers** → **Apache Tomcat**.

18. You should get the green indicator at *Apache Tomcat's* icon and *Apache Tomcat* output message with the time taken to start specified time to start *Apache Tomcat* web server.

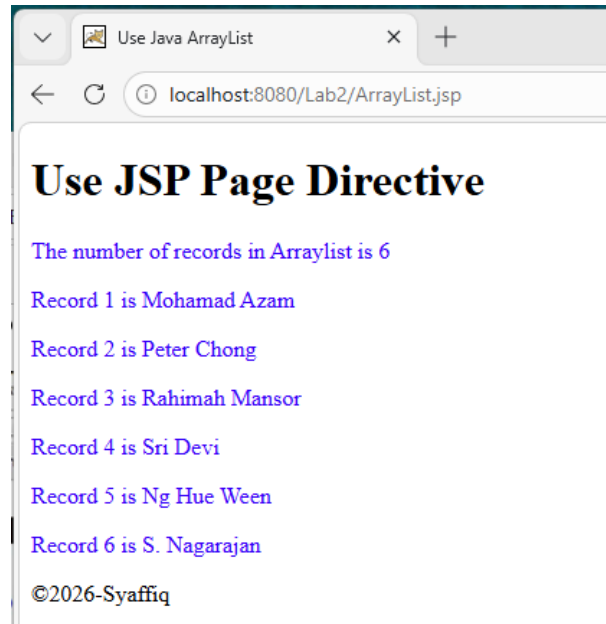


Figure 3.20: ArrayList Output

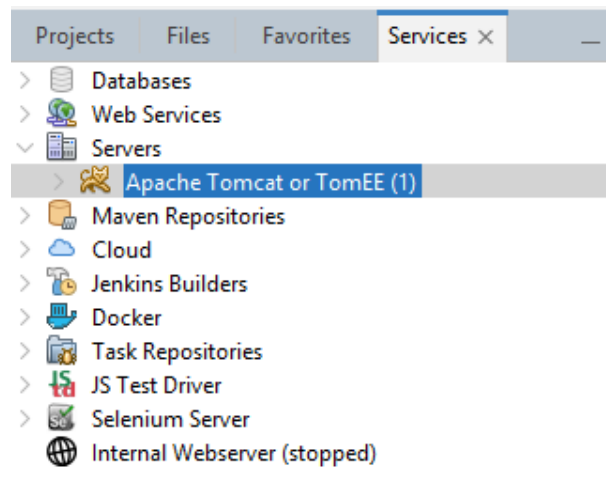


Figure 3.21: Tomcat Before Run

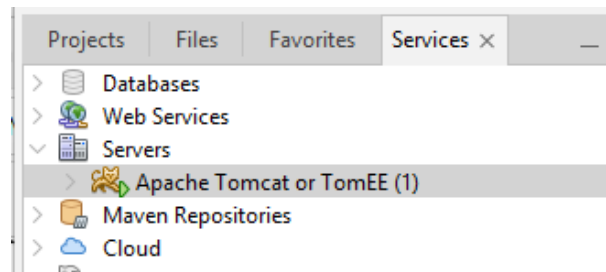
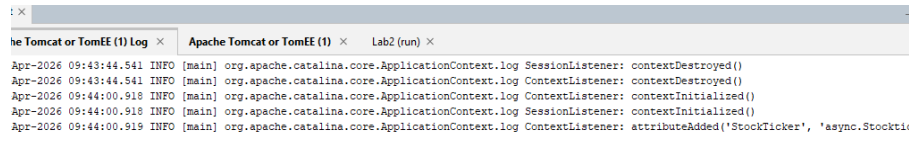


Figure 3.22: Tomcat After Run

Reflection

1. What you have learnt from this exercise?



```
Apr-2026 09:43:44.541 INFO [main] org.apache.catalina.core.ApplicationContext.log SessionListener: contextDestroyed()
Apr-2026 09:43:44.541 INFO [main] org.apache.catalina.core.ApplicationContext.log ContextListener: contextDestroyed()
Apr-2026 09:44:00.918 INFO [main] org.apache.catalina.core.ApplicationContext.log ContextListener: contextInitialized()
Apr-2026 09:44:00.918 INFO [main] org.apache.catalina.core.ApplicationContext.log SessionListener: contextInitialized()
Apr-2026 09:44:00.919 INFO [main] org.apache.catalina.core.ApplicationContext.log ContextListener: attributeAdded('StockTicker', 'async.Stocktic
```

Figure 3.23: Tomcat Success Message

2. Write a sample syntax how you want to use java *Math* object in JSP?
3. List and write a sample syntax for THREE (3) of JSP page directive.

3.9 Task 6: Use JSP Include directive for JSP Page

Objective: Demonstrate the use of JSP Page Include directive.

1. Create a new JSP's file.
2. Type file name as `mainPage`.
3. Create content for `mainPage.jsp` as below.

Using JSP Include directive

Java Server Page (JSP) is a technology for controlling the content or appearance of Web pages through the use of servlets, small programs that are specified in the Web page and run on the Web server to modify the Web page before it is sent to the user who requested it.

Figure 3.24: `mainPage.jsp`

4. Create a header file as `headerPage.jsp` and display the following output.

ABC Sdn Bhd

Figure 3.25: `headerPage.jsp` Output

5. Create a footer file as `footerPage.jsp` and display the following output.
6. Include your `headerPage.jsp` and `footerPage.jsp` inside your `mainPage.jsp`.
7. Save `mainPage.jsp`.
8. Compile and run `mainPage.jsp`.

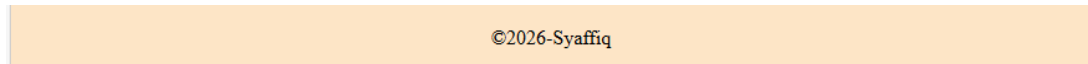


Figure 3.26: footerPage.jsp Output

9. You should get the following output



Figure 3.27: mainPage.jsp Output

Reflection

1. What you have learnt from this exercise?
2. Write a syntax how you want to include common.html file that located at a directory known as master.

3.9 Exercise

Objective: Develop a **Student Club Registration System** by integrating JSP Scriptlets, Page Directives, Include Directives, form handling, and dynamic data processing techniques learned throughout this lab.

Case Study

Faculty of Computer Science and Mathematics, UMT is organizing a **Student Club Recruitment Week** to allow students to register for extracurricular clubs and activities. As a web developer, you are required to build a simple JSP-based registration system to assist the organizing committee in managing student registrations and displaying activity information.

Instruction

Develop the system by fulfilling the following requirements:

1. Create a reusable **header.jsp** and **footer.jsp** for the system layout.
2. Apply the **JSP Include Directive** to include the header and footer in all JSP pages.
3. Design a navigation menu in the header that links to the following pages:
 - Registration Page
 - Fee Calculator Page
 - Club Member Directory
4. Create a **registration form** in `registerClub.jsp` that collects:
 - Student Name
 - Matric Number
 - Selected Club
5. Submit the registration form to `processRegistration.jsp` and display all submitted information using `request.getParameter()`.
6. Create a **membership fee calculator** in `feeCalculator.jsp` that:
 - Accepts the number of activities joined.
 - Calculates the total fee where each activity costs RM10.
 - Displays the final amount formatted to 2 decimal places.
7. Create a **club member directory page** in `memberDirectory.jsp` that:
 - Uses JSP Page Directive to import the required Java API.
 - Stores at least five committee member names using an **ArrayList**.
 - Dynamically displays the names in an HTML table.

Assessment Rubric (10 Marks)

Criteria	Marks
System Structure: Proper development of multiple JSP pages with logical navigation and reusable layout components.	2
Include Directive: Correct implementation of reusable header/footer using JSP Include Directive.	2
Form Handling: Correct form design and data retrieval using <code>request.getParameter()</code> .	2
Mathematical Logic: Accurate fee calculation and output formatting.	2
Dynamic Data Display: Proper use of JSP Page Directive and ArrayList to display dynamic table data.	2
Total	10